

WEEK 6 PRACTICAL

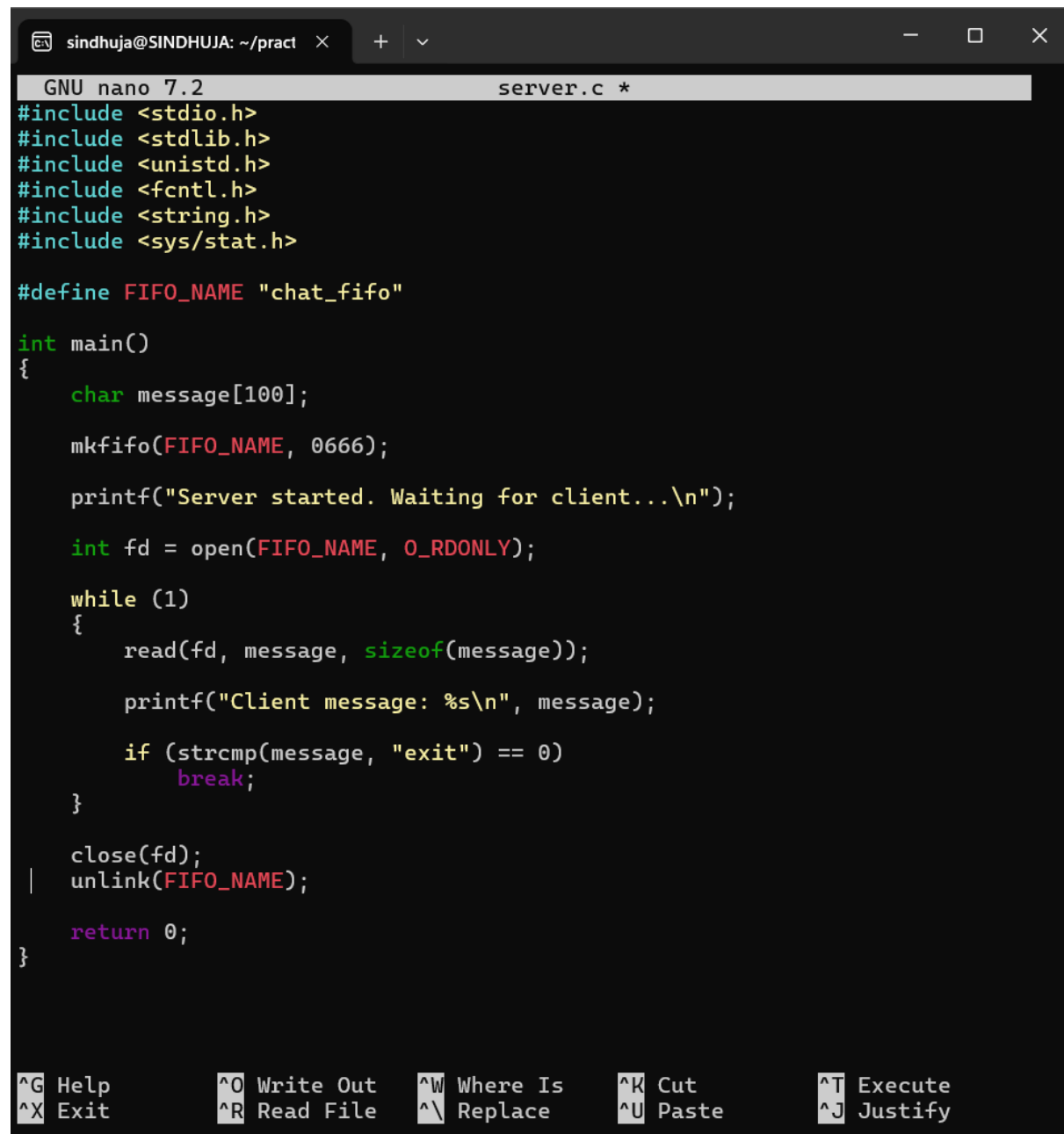
QUESTION 1

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Aim

To implement client-server communication using Named Pipes (FIFOs), where clients send messages to a server and the server processes and responds to them.

Program — Server (server.c)



```
GNU nano 7.2 server.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <sys/stat.h>

#define FIFO_NAME "chat_fifo"

int main()
{
    char message[100];

    mkfifo(FIFO_NAME, 0666);

    printf("Server started. Waiting for client...\n");

    int fd = open(FIFO_NAME, O_RDONLY);

    while (1)
    {
        read(fd, message, sizeof(message));

        printf("Client message: %s\n", message);

        if (strcmp(message, "exit") == 0)
            break;
    }

    close(fd);
    unlink(FIFO_NAME);

    return 0;
}
```

Help Write Out Where Is Cut Execute
Exit Read File Replace Paste Justify

Program — Client (client.c)



```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

#define FIFO_NAME "chat_fifo"

int main()
{
    char message[100];

    int fd = open(FIFO_NAME, O_WRONLY);

    while (1)
    {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        message[strcspn(message, "\n")] = '\0';

        write(fd, message, strlen(message) + 1);

        if (strcmp(message, "exit") == 0)
            break;
    }

    close(fd);
    return 0;
}
```

Terminal window showing the source code for client.c in nano editor. The window title is "sindhuja@SINDHUJA: ~". The code includes headers for stdio, stdlib, unistd, fcntl, and string. It defines a FIFO named "chat_fifo". The main function opens the FIFO for write-only access and enters a loop where it prompts the user for a message, reads it, and writes it to the FIFO. The loop breaks if the message is "exit". Finally, it closes the FIFO and returns 0. The bottom of the window shows nano editor shortcuts: ^G Help, ^X Exit, ^O Write Out, ^R Read File, ^W Where Is, ^\ Replace, ^K Cut, and ^U Paste.

Output

Terminal 1 — Server

```
sindhuja@SINDHUJA:~$ rm -rf practical_week6
sindhuja@SINDHUJA:~$ mkdir practical_week6
sindhuja@SINDHUJA:~$ cd practical_week6
sindhuja@SINDHUJA:~/practical_week6$ nano server.c
sindhuja@SINDHUJA:~/practical_week6$ gcc server.c -o server
sindhuja@SINDHUJA:~/practical_week6$ ./server
Server started. Waiting for client...
Client message: Hello Server
Client message: This is Client
Client message: exit
sindhuja@SINDHUJA:~/practical_week6$ |
```

Terminal 2 — Client

```
sindhuja@SINDHUJA: ~/pract x + v
sindhuja@SINDHUJA:~$ cd ~/practical_week6
sindhuja@SINDHUJA:~/practical_week6$ nano client.c
sindhuja@SINDHUJA:~/practical_week6$ gcc client.c -o client
sindhuja@SINDHUJA:~/practical_week6$ ./client
Enter message: Hello Server
Enter message: This is Client
Enter message: exit
sindhuja@SINDHUJA:~/practical_week6$ |
```

Observation

The named FIFOs provide communication between processes without requiring them to be related by `fork()`. The client writes messages to the client-to-server FIFO, while the server reads and processes them and sends responses through the server-to-client FIFO.

When multiple clients communicate simultaneously, messages are placed into the FIFO according to the order in which writes occur. A single FIFO does not automatically provide separate communication channels for individual clients, so a practical multi-client server would normally use separate response FIFOs or another mechanism to identify each client.

Result

The client-server communication system was successfully implemented using Named Pipes (FIFOs), and the behavior of FIFO communication with multiple clients was analyzed.

QUESTION 2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Aim

To implement POSIX signal handling using SIGINT, SIGTERM, and SIGUSR1, and demonstrate how signal handlers respond to asynchronous events.

Program



```
GNU nano 7.2 practical6_task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
        printf("\nSIGINT received (Ctrl + C)\n");

    else if (sig == SIGTERM)
        printf("\nSIGTERM received (Termination Signal)\n");

    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received (User Defined Signal)\n");
}

int main()
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Process ID (PID): %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1)
    {
        pause();
    }

    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Output

```
sindhuja@SINDHUJA:~/OSSP/Week6$ nano signal_handler.c
sindhuja@SINDHUJA:~/OSSP/Week6$ gcc signal_handler.c -o signal_handler
sindhuja@SINDHUJA:~/OSSP/Week6$ ./signal_handler
Process ID: 773
Waiting for signals...
Press Ctrl+C to send SIGINT.
Use 'kill -SIGUSR1 PID' to send SIGUSR1.
Use 'kill -SIGTERM PID' to send SIGTERM.
^C
SIGINT received. Interrupt signal handled.

SIGUSR1 received. User-defined signal handled.

SIGTERM received. Termination signal handled.
```

Observation

The program registers a signal handler for SIGINT, SIGTERM, and SIGUSR1. When any of these signals is delivered, the operating system asynchronously invokes the corresponding handler.

- **SIGINT** — normally generated using Ctrl+C.
- **SIGTERM** — requests termination of a process.
- **SIGUSR1** — a user-defined signal that can be used for application-specific events.

The pause() system call keeps the process waiting until a signal is received.

Result

The POSIX signal handling program was successfully implemented. The program captured SIGINT, SIGTERM, and SIGUSR1 and demonstrated asynchronous event handling using signal handlers.